

Roteiro da Apresentação P2

EasyGis — Distribuição de slides e falas por integrante

Engenharia de Software · 2026.1

Maio · 2026

Visão geral

A apresentação tem **14 slides** cobrindo o ciclo completo do produto: problema, solução, escopo, arquitetura, fluxo técnico, demo ao vivo, testes, resultados, limitações e equipe. Tempo previsto: **17–18 minutos**.

A equipe é dividida da seguinte forma:

#	Slide	Apresentador
1	Capa	Lucas Benevinuto
2	Problema	Lucas Benevinuto
3	Solução	Lucas Benevinuto
4	Para quem	João Vinícius Castello
5	Backlog (30/35 entregues)	José Melquíades
6	Arquitetura	Vinicius Henrique
7	Stack tecnológica	Luis Gustavo
8	Decisões de design (ADRs)	Luis Gustavo
9	Fluxo do NDVI	João Leonardi
10	Demonstração ao vivo	João Vinícius Castello (condutor)
11	Estratégia de testes	Lauan Matheus
12	Resultados (28/28, cobertura)	Lauan Matheus
13	Limitações e próximos passos	Sammuel Moura
14	Equipe e Perguntas?	João Leonardi (encerra) + todos

Resumo por integrante:

- **Lucas Benevinuto** — slides 1, 2, 3 (abertura + problema + solução)
- **João V. Castello** — slides 4, 10 (perfis + condução da demo)
- **José Melquíades** — slide 5 (backlog)
- **Vinicius Henrique** — slide 6 (arquitetura)
- **Luis Gustavo** — slides 7, 8 (stack + ADRs)
- **João Leonardi** — slides 9, 14 (fluxo NDVI + encerramento)
- **Lauan Matheus** — slides 11, 12 (testes + resultados)
- **Sammuel Moura** — slide 13 (limitações + próximos passos)

Falas — slide a slide

Slide 1 — Capa

Lucas Benevinuto · 30 s

“Boa tarde, professor. Somos a equipe que apresenta o **EasyGis** — uma plataforma web de agricultura de precisão por sensoriamento remoto. Nosso trabalho foi entregar uma ferramenta que transforma imagens Sentinel-2 em decisões sobre uma lavoura, sem custo para o produtor. Eu sou o Lucas, vou conduzir a abertura.”

Slide 2 — Problema

Lucas Benevinuto · 60 s

“Para entender o porquê do projeto, três números: o Brasil tem **66 milhões de hectares** plantados, a Agência Espacial Europeia disponibiliza imagens **Sentinel-2** de **10 m de resolução a cada 5 dias, de graça**. Apesar disso, ferramentas prontas que usam esses dados custam **milhares de reais por ano** ou exigem que o usuário monte uma stack de GDAL, scripts e SIG no desktop. O EasyGis ataca exatamente esse gap: aproximar essa imagem gratuita do produtor.”

Slide 3 — Solução

Lucas Benevinuto · 50 s

“A solução é uma **página web local, sem custo**. O fluxo é direto: o usuário **importa um KML** do Google Earth ou desenha o talhão no mapa; escolhe o **índice** (NDVI, EVI, SAVI, NDWI, NDBI ou bandas brutas); e em **5 a 30 segundos** aparece a imagem colorida sobreposta ao mapa, com estatísticas e classificação em zonas de vigor com área em hectares. Tudo isso em uma única tela. Quem vai detalhar para quem isso serve é o João Castello.”

Slide 4 — Para quem

João V. Castello · 60 s

“O EasyGis foi pensado para **quatro perfis** muito diferentes, mas que olham para o mesmo mapa. O **produtor rural** quer saber onde aplicar amanhã. O **agrônomo** precisa de um mapa de variabilidade para prescrever insumos. O **pesquisador** usa a ferramenta como sandbox para validar algoritmos contra dados reais. E o **estudante** aprende sensoriamento remoto na prática, sem barreira de licença ou setup. Mesmo produto, leituras diferentes — e isso guiou nossa priorização. Vou passar para o José falar sobre escopo.”

Slide 5 — Backlog

José Melquíades · 80 s

“Sobre o escopo: começamos com **35 histórias distribuídas em 6 épicos** — importação, índices, visualização, classificação, 3D e infraestrutura. Priorizamos por MoSCoW: *Must Have*, *Should*, *Could* e *Won't Have*. Entregamos **28 histórias completas** e **2 parciais**, o que dá **85,7% do escopo planejado**, dentro do prazo do semestre. Os **5 *Won't Have*** foram **conscientes**: autenticação, persistência em banco, exportação e multi-usuário ficaram fora porque o foco era validar o **núcleo de processamento** geoespacial — e isso foi escolha, não acidente.”

Slide 6 — Arquitetura

Vinicius Henrique · 75 s

“Arquitetura: optamos por **dois processos separados**. O **frontend** é um Next.js 16 rodando na porta 3000 — React 19, TypeScript, Tailwind, Leaflet em 2D e Three.js para o relevo em 3D. O **backend** é uma API FastAPI em Python 3.12 na porta 8000, com rasterio, NumPy, SciPy, OpenCV e scikit-learn. A comunicação entre eles é **REST com JSON**; as imagens processadas viajam em **base64** dentro da resposta. **Não há banco de dados** — os KMLs e os produtos Sentinel ficam no filesystem. Essa decisão está registrada como **DA-02** no documento de ADRs e é o que torna o sistema reproduzível: qualquer máquina com Python e Node sobe a aplicação em minutos.”

Slide 7 — Stack tecnológica

Luis Gustavo · 75 s

“Cada peça da stack tem uma razão. **FastAPI** porque o Pydantic dá validação de contrato automática e gera a doc OpenAPI em /docs de graça. **rasterio** porque é o padrão de fato sobre o GDAL no ecossistema Python. **Next.js App Router** porque combina SSR e API routes no mesmo projeto, sem expor o filesystem ao cliente. **Leaflet** em vez de Mapbox porque não há custo por tile carregado. **Three.js** para visualização 3D nativa em WebGL. E **uv** no Python — gerenciador de pacotes **dez vezes mais rápido que o pip**, com lockfile determinístico para garantir reprodução em qualquer máquina. Nada foi escolhido por hype — toda decisão está justificada.”

Slide 8 — Decisões de design

Luis Gustavo · 80 s

“Documentamos **11 ADRs** no repositório, e quero destacar três que são chave para entender por que o sistema é robusto.

DA-06 — Separação processador / rotas HTTP. A lógica de cálculo dos índices vive numa classe **SentinelProcessor** que **não conhece HTTP**. Isso permite testar a

matemática sem subir servidor — são 28 testes unitários em menos de 1 segundo.

DA-07 — Tipagem fim-a-fim. O Pydantic no backend é espelhado por interfaces TypeScript no frontend. Um erro de contrato vira **422 ou erro de build**, nunca **500 silencioso** em produção.

DA-09 — Suavização gaussiana preservando NaN. Um filtro gaussiano direto **sangra valores** para fora do polígono e cria halo nas bordas. Resolvemos dividindo também pela máscara binária — o detalhe técnico que separa o resultado correto do plausível-mas-errado.”

Slide 9 — Fluxo do NDVI

João Leonardi · 75 s

“Para mostrar como tudo isso se encadeia, vamos pelo caminho do NDVI em **5 etapas**.

(01) O usuário seleciona o talhão e o índice no frontend. **(02)** O Next.js dispara `POST /calculate-index` para o backend. **(03)** O `SentinelProcessor` usa rasterio para ler as bandas **B08 (infravermelho próximo)** e **B04 (vermelho)** do produto `.SAFE`. **(04)** O `pyproj` reprojeta do WGS84 do KML para o UTM do raster. **(05)** NumPy faz o cálculo elemento a elemento e o Pillow gera o PNG colorido, que volta em base64 e o Leaflet sobrepõe ao mapa.

A fórmula é $\text{NDVI} = (\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red})$, clipada em $[-1, 1]$. O gradiente vai de marrom (solo) para amarelo (estresse) até verde (vegetação saudável).”

Slide 10 — Demonstração

João V. Castello · 3–4 min

“Agora vamos ver isso funcionando. Eu vou conduzir a demo enquanto a equipe complementa.”

Roteiro da demo:

- 01** Carregar o talhão `crop_field_1` em Picos-PI.
- 02** Visualizar o **NDVI** — overlay no mapa, estatísticas e histograma.
- 03** Clicar em um pixel — popup com nível de estresse na coordenada.
- 04** Trocar para **EVI** e comparar lado a lado.
- 05** Ativar o **terreno 3D** e rotacionar (Three.js).
- 06** Acionar a **classificação em zonas** (K-Means) — hectares por classe.

Fala de fechamento da demo: ”Tudo isso roda localmente, sem login, sem nuvem, e em alguns segundos por imagem. Vou passar para o Lauan falar sobre testes.”

Slide 11 — Estratégia de testes

Lauan Matheus · 75 s

“Sobre testes, montamos uma **pirâmide em três níveis**.

Nível 1 — Unitário. pytest direto no `SentinelProcessor`: NDVI, EVI, estatísticas, casos de borda como NaN, divisão por zero e máscara de água. Esse nível roda em milissegundos.

Nível 2 — Integração. FastAPI TestClient valida os endpoints HTTP, os status codes e a validação Pydantic do contrato.

Nível 3 — Manual. Testes end-to-end com um produto Sentinel-2 real, executados durante a demonstração que vocês acabaram de ver.

Tudo isso é viabilizado por **fixtures sintéticas em NumPy**, que substituem um produto Sentinel real de 1 GB. A suíte completa roda em **menos de 1 segundo**.”

Slide 12 — Resultados

Lauan Matheus · 75 s

“Os números: **28 de 28 testes passando em 0,79 segundos**, com **zero flaky**. A cobertura está dividida por camada: **55%** no `sentinel_processor`, que é onde a lógica de cálculo mora; e **24%** global, porque os endpoints exigem um produto Sentinel real para serem exercitados.

E isso é **honesto**: declaramos esse gap como **dívida técnica DT-03** no documento de entrega. Rastreabilidade: os requisitos **RF-04, 05, 06, 10, 11 e 20** estão diretamente cobertos por casos de teste vinculados.”

Slide 13 — Limitações + próximos passos *Sammuel Moura · 90*

s

“Encerro o bloco técnico com honestidade sobre **limitações conscientes**:

- Talhões desenhados **não persistem** — o estado é em memória.
- A classificação supervisionada está **simulada** (K-Means + heurística).
- As imagens Sentinel-2 são **baixadas manualmente** do Copernicus Hub.
- Aplicação **single-user** em localhost, sem autenticação.
- A cobertura de I/O depende de produto **.SAFE** real.

E **sete próximos passos** concretos: **(01)** integração com a API do Copernicus para baixar imagem sob demanda; **(02)** persistência em SQLite ou PostGIS; **(03)** Random Forest para classificação real; **(04)** exportação em GeoTIFF, Shapefile e CSV; **(05)**

comparação temporal multi-data; **(06)** hospedar a aplicação para multi-usuário; e **(07)** CI automatizado com pytest + GitHub Actions.

Vou passar para o João Leonardi encerrar. ”

Slide 14 — Equipe + Perguntas

João Leonardi · 60 s

“Para fechar, a divisão de trabalho da equipe: eu fiquei no **backend FastAPI e integração Git**; João Castello no **frontend e na condução desta demo**; José Melquíades na **modelagem de dados e parsing KML**; Lauan Matheus na **suíte de testes**; Lucas Benevinuto na **apresentação**; Luis Gustavo na **documentação técnica e ADRs**; Samuel Moura no **manual do usuário e glossário**; e Vinicius Henrique em **arquitetura, DevOps e roteiro de demo**.

Obrigado pela atenção. Estamos abertos a perguntas. ”

Dicas de execução

- **Transições.** Cada apresentador encerra com uma frase que **passa a bola** explicitamente (*“vou passar para o X falar sobre Y”*). Evita pausas mortas e cola o pitch.
- **Demo (slide 10).** O condutor deve ter o produto **pré-aberto e com o KML já carregado**. O risco maior é gastar tempo procurando arquivo na hora.
- **Números que vocês precisam saber de cabeça:** **35 histórias (30 entregues, 85,7%), 11 ADRs, 28/28 testes em 0,79 s, cobertura 55% processador / 24% global, 6 épicos, 5 *Won't Have*** conscientes.
- **Se perguntarem por que não tem banco:** a resposta está em DA-02 — comunicação por JSON+base64, sem persistência, para manter o sistema reprodutível e single-user em localhost.
- **Se perguntarem sobre cobertura baixa de 24%:** é um número honesto — declarado como DT-03 — e é consequência direta dos endpoints precisarem de um produto Sentinel .SAFE real, que tem 1 GB. O que importa é os **55% no processador**, que é onde a lógica de cálculo vive.
- **Ritmo.** Mantenham cerca de **1 minuto por slide** (exceto a demo). Se ficarem atrasados, é melhor abreviar o slide 7 (Stack) — é o mais denso e menos crítico para a banca.